

VO 182.711 / 182.737
Prüfung Betriebssysteme

28. Januar 2019

KNr.

MNr.

Zuname, Vorname

Studium:

☐ Informatik (Bakk.)

☐ Elektrotechnik (Master)

Zusatzblätter:

Ges.)(100)

1.)(30)

2.)(30)

3.)(40)

Bitte verwenden Sie nur dokumentenechtes Schreibmaterial!

1 Synchronisation mit Semaphoren (30)

Mit Hilfe der Prozesse *Schiff1*, *Schiff2* und *Passagier* soll der Betrieb von zwei Rundfahrtschiffen simuliert werden.

- *Schiff1*, *Schiff2*: Jeder dieser Prozesse wird genau einmal gestartet und simuliert eines der Schiffe. Die Schiffe fahren abwechselnd. Für jede Rundfahrt lassen sie Passagiere an der Anlegestelle einsteigen, fahren eine Runde und lassen dann die Passagiere wieder aussteigen.
Der Prozess *SchiffX* ($X \in \{1, 2\}$) simuliert das Öffnen und Schließen des Schiffszugangs zum Ein- und Aussteigen an der Anlegestelle durch Aufrufe der Funktionen *zugang_auf()* bzw. *zugang_zu()*. Die Rundfahrt erfolgt durch Aufruf der Funktion *rundfahrt()*.
- *Passagier*: Von diesem Prozess kann es beliebig viele Instanzen im System geben. Er simuliert einen Passagier bei der Schifffahrt.
Die Funktionen *einsteigen()* und *aussteigen()* simulieren das Einsteigen in bzw. Aussteigen aus dem gerade verfügbaren Schiff.

Ergänzen Sie die auf der folgenden Seite gegebenen Prozesstemplates ausschließlich mit **Semaphoroperationen** (die Verwendung von globalen Variablen ist nicht erlaubt), um die Prozesse zu synchronisieren. Lokale Variable und Schleifen dürfen Sie verwenden, wenn Sie diese für Ihre Lösung benötigen. Weiters sind folgende Punkte zu beachten:

- Jedes Schiff kann N Fahrgäste aufnehmen. Das Schiff fährt vom Anlegeplatz ab, wenn es voll ist. Das Einsteigen von Passagieren beginnt, sobald alle Passagiere der vorigen Rundfahrt das Schiff verlassen haben und das andere Schiff voll mit Fahrgästen beladen ist.
- Beim Ein- und Aussteigen kann der Zugang zum Schiff immer nur von einem Passagier gleichzeitig betreten werden.

Initialisierungen

```
init (S1, 1);  
init (S2, 0);  
init (x, 0);  
init (y, 0);
```

```
/** Code Schiff1 **/
```

```
for(;;) {
```

```
    wait (S1);
```

```
    zugang_auf();
```

```
    for (int i=0; i < N; i++) {  
        signal (x);  
        wait (y);  
    }
```

```
    zugang_zu();
```

```
    rundfahrt();
```

```
    zugang_auf();
```

```
    for (int i=0; i < N; i++) {  
        signal (x);  
        wait (y);  
    }
```

```
    zugang_zu();
```

```
    signal (S2);
```

```
}
```

```
/** Code Schiff2 **/
```

```
for(;;) {
```

```
    wait (S2);
```

```
    zugang_auf();
```

```
    for (int i=0; i < N; i++) {  
        signal (x);  
        wait (y);  
    }
```

```
    zugang_zu();
```

```
    rundfahrt();
```

```
    zugang_auf();
```

```
    for (int i=0; i < N; i++) {  
        signal (x);  
        wait (y);  
    }
```

```
    zugang_zu();
```

```
    signal (S1);
```

```
}
```

```
/** Code Passagier **/
```

```
    wait (x);
```

```
    einsteigen();
```

```
    signal (y);
```

```
/* geniessen */
```

```
    wait (x);
```

```
    aussteigen();
```

```
    signal (y);
```

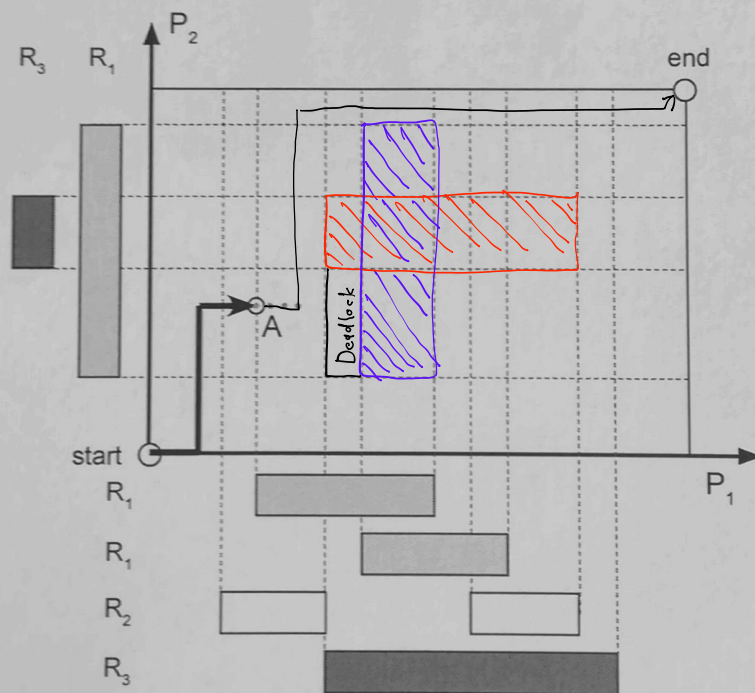
2 Deadlock (30)

Gegeben sind zwei Prozesse, P_1 und P_2 , die Ressourcen der Kategorien R_1 (2 Stück), R_2 (ein Stück) und R_3 (ein Stück) unter Mutual Exclusion verwenden.

Weiters zeigt ein Prozessfortschrittsdiagramm die Ressourcenbelegung von P_1 und P_2 während ihrer Abarbeitung, sowie den Teil einer (quasi)parallelen Abarbeitung der Prozesse als Kantenzug zwischen den Punkten *start* und *A*.

Teilaufgabe A (Prozessfortschrittsdiagramm)

1. Umranden und schraffieren Sie im Fortschrittsdiagramm jene Bereiche, durch die der Kantenzug einer (quasi)parallelen Abarbeitung wegen einer Ressourcenüberschreitung nicht möglich ist.
2. Kennzeichnen Sie auf unterschiedliche Weise die Bereiche, die von einem Kantenzug bei einer deadlockfreien Abarbeitung von P_1 und P_2 nicht passiert werden dürfen. Markieren Sie diesen Bereich deutlich mit dem Wort „Deadlock“.
3. Setzen Sie den gegebenen Kantenzug im Punkt *A* fort, sodass er eine gültige Abarbeitung von P_1 und P_2 darstellt.



Teilaufgabe B (Deadlock Avoidance)

Stellen Sie mit Hilfe des *Banker's Algorithm* fest, ob die durch den Kantenzug gegebene teilweise Abarbeitung der beiden Prozesse in Punkt A durch Fortführung des Prozesses P_1 (d.h., durch die Zuweisung einer Ressource der Kategorie R_1 an P_1) deadlockfrei fortgesetzt werden kann. Erstellen Sie die notwendigen Vektoren und Matrizen und wenden Sie den Banker's Algorithmus an. Beantworten Sie, ob P_1 am Punkt A fortgesetzt werden kann.

$$\begin{array}{llll} R: (2, 1, 1) & C: \begin{pmatrix} 2, 1, 1 \\ 1, 0, 1 \end{pmatrix} & A: \begin{pmatrix} 0, 1, 0 \\ 1, 0, 0 \end{pmatrix} & N: \begin{pmatrix} 2, 0, 1 \\ 0, 0, 1 \end{pmatrix} \\ V: (1, 0, 1) & & & \\ Q_1: (1, 0, 0) & & & \end{array}$$

$$W: (0, 0, 1) \quad C: \begin{pmatrix} 2, 1, 1 \\ 1, 0, 1 \end{pmatrix} \quad A: \begin{pmatrix} 1, 1, 0 \\ 1, 0, 0 \end{pmatrix} \quad N: \begin{pmatrix} 1, 0, 1 \\ 0, 0, 1 \end{pmatrix} \Leftarrow$$

$$W: (1, 0, 1) \quad C: \begin{pmatrix} 2, 1, 1 \\ 1, 0, 1 \end{pmatrix} \quad A: \begin{pmatrix} 1, 1, 0 \\ 0, 0, 0 \end{pmatrix} \quad N: \begin{pmatrix} 1, 0, 1 \\ 0, 0, 0 \end{pmatrix} \Leftarrow$$

$$W: (2, 1, 1) \quad C: \begin{pmatrix} 2, 1, 1 \\ 1, 0, 1 \end{pmatrix} \quad A: \begin{pmatrix} 0, 0, 0 \\ 0, 0, 0 \end{pmatrix} \quad N: \begin{pmatrix} 0, 0, 0 \\ 0, 0, 0 \end{pmatrix} \Leftarrow$$

↓
safe state